Object-Relational Mapping

Parte 1 - Entidades, Atributos Simples e Herança

JPA

Java Persistence API

- Standard em Java para mapear objectos POJO no modelo relacional.
- javax.persistence
- O código (Java orientado a objectos) é complementado com metadados. Podem ser ficheiros de configurações ou anotações.

(Plain Old Java Objects = POJOs)

JPA Annotations

java.lang

Annotation Type SuppressWarnings

```
@Target(value={TYPE,FIELD,METHOD,PARAMETER,CONSTRUCTOR,LOCAL_VARIABLE})
@Retention(value=SOURCE)
public @interface SuppressWarnings
```

Required Elements

Modifier and Type	Required Element and Description
`String[]`	`value` The set of warnings that are to be suppressed by the compiler in the annotated element.

```
@Target({TYPE, FIELD, METHOD, PARAMETER, CONSTRUCTOR, LOCAL_VARIABLE})
@Retention(RetentionPolicy.SOURCE)
public @interface SuppressWarnings {
    * The set of warnings that are to be suppressed by the compiler in the
    String[] value();
}
```

```
@SuppressWarnings(value = "unchecked")
void myMethod() { ... }
```

```
@SuppressWarnings("unused")
private String description;
```

Anotações

- Forma de codificar meta-dados no código.
- Podem ser usados em análise estática ou dinâmica, alterando os comportamentos normais de Java.
- Diferentes versões existem em várias linguagens (C, Rust, Java, Python)

```
@SuppressWarnings("unused")
private String description;
```

Anotações em Java

- Sobre classes, métodos, atributos, parâmetros
- Têm argumentos (opcionais)
- @interface

```
@SuppressWarnings("unused")
private String description;
```

```java
@interface Example{
    //Precisa ser chamado value()!
    int value();
}

public class AnnotationExample {
    public static void main(String[] args) {
        @Example(33)  //sem atribuição explícita
        String name;
    }
}
```

```java
@interface Example{
    String ocupacao() default "Professor";
    int value();
}

public class AnnotationExample {
    public static void main(String[] args) {
        @Example(33)
        String name;
    }
}
```

```java
@interface Example{
    String ocupacao() default "Professor";
    int age() default 35;
}

public class AnnotationExample {
    public static void main(String[] args) {
        @Example()
        String name;
    }
}
```

```java
@interface Example{
    String ocupacao() default "Professor";
    int age() default 35;
}

public class AnnotationExample {
    public static void main(String[] args) {
        @Example()
        String name;
    }
}
```

```java
import java.lang.reflect.Field;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;


@Retention(RetentionPolicy.CLASS)
@interface Example { String ocupacao(); }

@Retention(RetentionPolicy.RUNTIME)
@interface Example2 { String ocupacao(); }

@Retention(RetentionPolicy.SOURCE)
@interface Example3 { String ocupacao(); }

public class AnnotationExample {
    @Example(ocupacao = "Professor")
    String Wellington;

    @Example2(ocupacao = "Aluno")
    String Frederico;

    @Example3(ocupacao = "Cientista")
    String MarieCurie;
```

- Até que ponto no ciclo de vida vamos preservar a anotação

```java
import java.lang.reflect.Field;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;


@Retention(RetentionPolicy.CLASS)
@interface Example { String ocupacao(); }

@Retention(RetentionPolicy.RUNTIME)
@interface Example2 { String ocupacao(); }

@Retention(RetentionPolicy.SOURCE)
@interface Example3 { String ocupacao(); }

public class AnnotationExample {
    @Example(ocupacao = "Professor")
    String Wellington;

    @Example2(ocupacao = "Aluno")
    String Frederico;

    @Example3(ocupacao = "Cientista")
    String MarieCurie;
```

- Fica disponível no .class mas não é possível aceder em tempo de execução
  - *default* do Java

```java
import java.lang.reflect.Field;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;


@Retention(RetentionPolicy.CLASS)
@interface Example { String ocupacao(); }

@Retention(RetentionPolicy.RUNTIME)
@interface Example2 { String ocupacao(); }

@Retention(RetentionPolicy.SOURCE)
@interface Example3 { String ocupacao(); }

public class AnnotationExample {
    @Example(ocupacao = "Professor")
    String Wellington;

    @Example2(ocupacao = "Aluno")
    String Frederico;

    @Example3(ocupacao = "Cientista")
    String MarieCurie;
```

- Também no .class mas fica disponível **em tempo de execução**
  - ferramentas ou frameworks podem acessem através de reflexão.

```java
import java.lang.reflect.Field;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;


@Retention(RetentionPolicy.CLASS)
@interface Example { String ocupacao(); }

@Retention(RetentionPolicy.RUNTIME)
@interface Example2 { String ocupacao(); }

@Retention(RetentionPolicy.SOURCE)
@interface Example3 { String ocupacao(); }

public class AnnotationExample {
    @Example(ocupacao = "Professor")
    String Wellington;

    @Example2(ocupacao = "Aluno")
    String Frederico;

    @Example3(ocupacao = "Cientista")
    String MarieCurie;
```

- Só existe no código fonte
  - descartado na compilação
  - não está no .class.

```java
@Retention(RetentionPolicy.CLASS)
@interface Example { String ocupacao(); }

@Retention(RetentionPolicy.RUNTIME)
@interface Example2 { String ocupacao(); }

@Retention(RetentionPolicy.SOURCE)
@interface Example3 { String ocupacao(); }

public class AnnotationExample {
    @Example(ocupacao = "Professor")
    String Wellington;

    @Example2(ocupacao = "Aluno")
    String Frederico;

    @Example3(ocupacao = "Cientista")
    String MarieCurie;

    public static void main(String[] args) throws Exception {
        AnnotationExample obj = new AnnotationExample();

        Field f1 = AnnotationExample.class.getDeclaredField("Wellington");
        System.out.println(f1.getAnnotation(Example.class));

        Field f2 = AnnotationExample.class.getDeclaredField("Frederico");
        System.out.println(f2.getAnnotation(Example2.class));

        Field f3 = AnnotationExample.class.getDeclaredField("MarieCurie");
        System.out.println(f3.getAnnotation(Example3.class));
    }
}
```

?

Mapeamento Classe - Tabela

- Classes anotadas com @Entity são mapeadas para uma tabela.
- JPA tem um comportamento para todos os mapeamentos por omissão. Excepções são indicadas com anotações:
  - @Table muda o nome da tabela na BD
- Entities **precisam** de constructor publico/protected sem argumentos

Mapeamento Classe - Tabela

Desafios

- Referências vs chaves primárias
- **Mapeamento de atributos**
- Mapeamento de relações
- Mapeamento de herança

Mapeamento Classe - Tabela

Identity Field

- Campo imutável para a chave primária
- Campo **sem significado lógico**
  - Não usar CC, NIF, etc...
- Preenchido pelo ORM a partir da BD.

Mapeamento Classe - Tabela

Referências vs chaves primárias

- @Id marca como Chave Primária
- @GeneratedValue define a estratégia (default=auto)

```
public class Person {
    @Id @GeneratedValue(strategy = GenerationType.TABLE)
    private int id;
```

- Int ou long (preferível)

Mapeamento Classe - Tabela

Referências vs chaves primárias

javax.persistence

Enum GenerationType

AUTO
Indicates that the persistence provider should pick an appropriate strategy for the particular database.

IDENTITY
Indicates that the persistence provider must assign primary keys for the entity using a database identity column.

SEQUENCE
Indicates that the persistence provider must assign primary keys for the entity using a database sequence.

TABLE
Indicates that the persistence provider must assign primary keys for the entity using an underlying database table to ensure uniqueness.

Estratégia dependente do SGBD!

Mapeamento de Atributos

Tipos básicos

- `int → Integer`
- `double → Real`
- `char → Character`
- `string → VarChar`

- Mapeamento automático, dependente do SGBD

- @Basic permite fazer override.

Mapeamento de Atributos

Propriedades temporais

```
@Column(name = "local_time", columnDefinition = "TIME")
private LocalTime localTime;

@Column(name = "local_date", columnDefinition = "DATE")
private LocalDate localDate;

@Column(name = "local_date_time", columnDefinition = "TIMESTAMP")
private LocalDateTime localDateTime;
```

Mapeamento de Atributos

Enumerados

```
@Entity
public class Article {
    @Id
    private int id;

    private String title;

    @Enumerated(EnumType.ORDINAL)
    private Status status;

    @Enumerated(EnumType.STRING)
    private Type type;
}
```

Guardar na BD como:

- número
  - +eficiente, -estável
- string
  - -eficience, +estável

Mapeamento de Atributos

Enumerados

```
@Entity
public class Article {
    @Id
    private int id;

    private String title;

    @Enumerated(EnumType.ORDINAL)
    private Status status;

    @Enumerated(EnumType.STRING)
    private Type type;
}
```

Guardar na BD como:

- número
  - +eficiente, -estável
- string
  - -eficience, +estável

```
public enum Status {
    CLOSED, OPEN, CANCELLED;
}
```

0 1 2

Mapeamento de Atributos

Enumerados

```java
@Entity
public class Article {
    @Id
    private int id;

    private String title;

    @Enumerated(EnumType.ORDINAL)
    private Status status;

    @Enumerated(EnumType.STRING)
    private Type type;
}
```

Guardar na BD como:

- número
  - +eficiente, -estável
- string
  - -eficience, +estável

```java
public enum Status {
    CLOSED, OPEN, CANCELLED;
}
```

0 1 2

```java
public enum Status {
    CLOSED, RETURNED, OPEN, CANCELLED;
}
```

0 1 2 3

Mapeamento de Atributos

Agregação de dados

```java
@Entity
public class Employee {
  @Id
  @Column(name="EMP_ID")
  private long id;
  ...
  @ElementCollection
  @CollectionTable(
        name="PHONE",
        joinColumns=@JoinColumn(name="OWNER_ID")
  )
  private List<Phone> phones;
  ...
}
```

```java
@Embeddable
public class Phone {
  private String type;
  private String areaCode;
  @Column(name="P_NUMBER")
  private String number;
  ...
}
```

Padrão Dependent Mapping:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega

Padrão Embeddable Value:

- Tipos personalizados necessitam do @Embeddable, quando não são entidades: são apenas campos compostos
- @Embedded no dono, quando é 1-1

Mapeamento de Atributos

Agregação de dados

```java
@Entity
public class Employee {
  @Id
  @Column(name="EMP_ID")
  private long id;
  ...
  @ElementCollection
  @CollectionTable(
        name="PHONE",
        joinColumns=@JoinColumn(name="OWNER_ID")
  )
  private List<Phone> phones;
  ...
}
```

```java
@Embeddable
public class Phone {
  private String type;
  private String areaCode;
  @Column(name="P_NUMBER")
  private String number;
  ...
}
```

Padrão Dependent Mapping:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega

Padrão Embeddable Value:

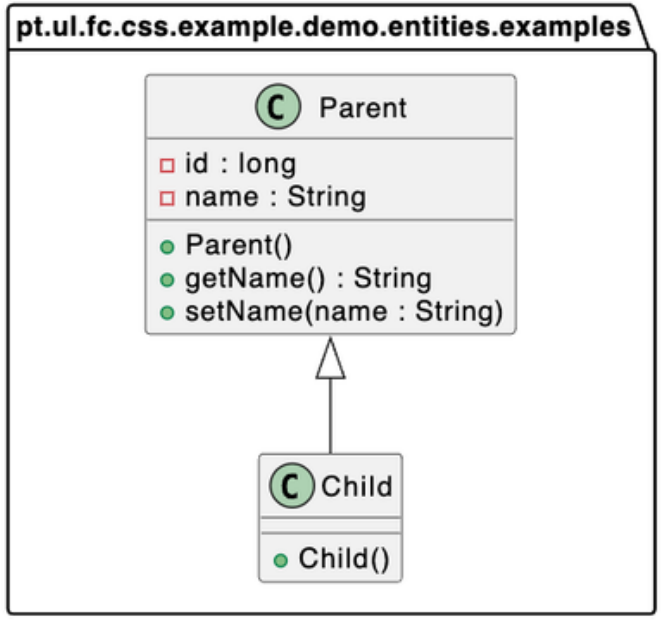
- Tipos personalizados necessitam do @Embeddable, quando não são entidades: são apenas campos compostos
- @Embedded no dono, quando é 1-1

@ElementCollection quando é 1-N
@Embedded quando é 1-1

Herança

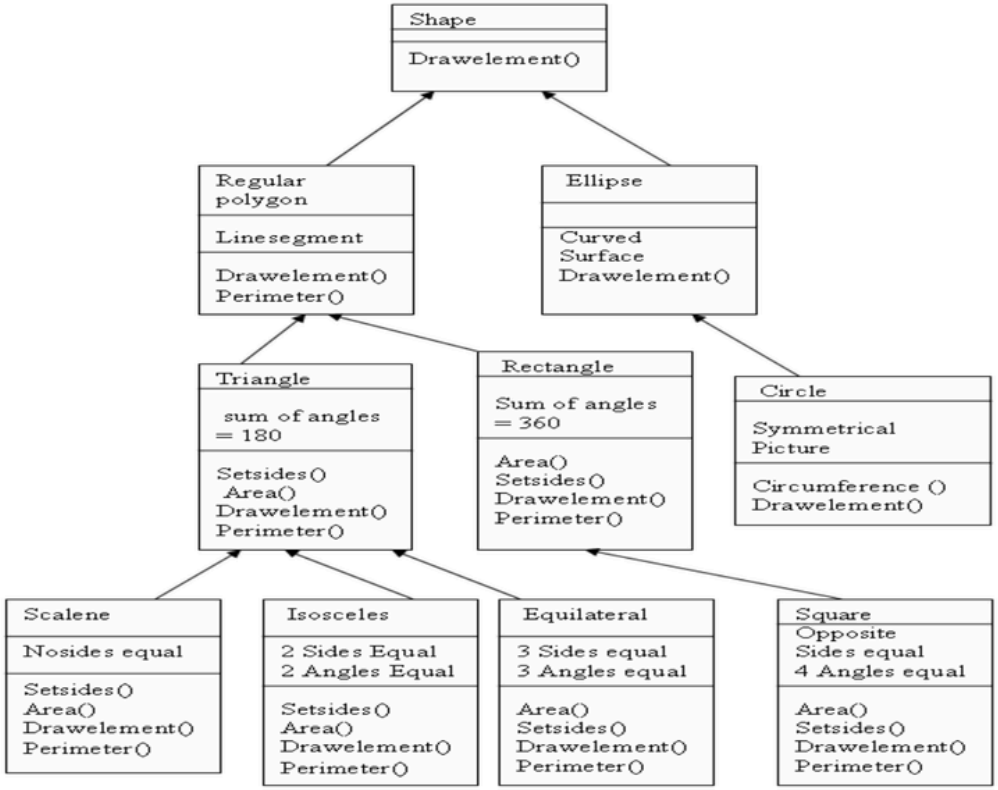
Conceito de OOP

Problema



- A classe filha não tem atributos adicionais. Como mapear no modelo relacional?

Problema



- Temos uma hierarquia com 10 níveis de subclasses. Como mapear essas relações em tabelas?
- E como lidar com classes abstractas?

Problema

- O modelo relacional não tem herança!
- Temos de mapear a herança de alguma forma...

Solução

- Em cada hierarquia de classes, definir como é feito o mapeamento, escolhendo de alguns padrões standard.

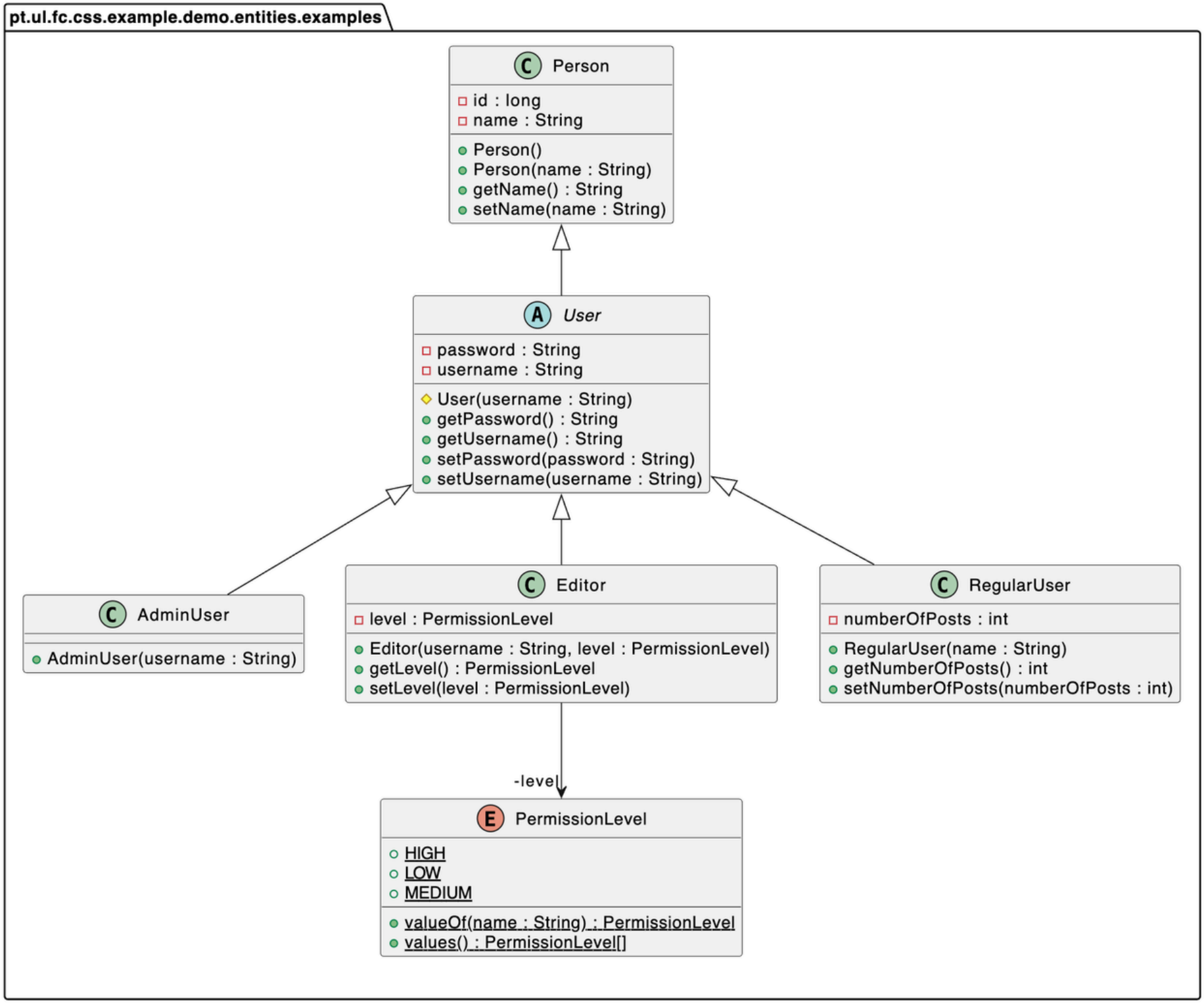
3 alternativas

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Parent {
```

Anotação é colocada na classe mãe.

*Although the details of this behavior vary with your inheritance mapping scheme (**Single Table Inheritance** (278), **Class Table Inheritance** (285), and **Concrete Table Inheritance** (293)) the general structure works the same for all of them.*

Exemplo


pt.ul.fc.css.example.demo.entities.examples
Person
id : long
name : String
Person()
Person(name : String)
getName() : String
setName(name : String)
User
password : String
username : String
User(username : String)
getPassword() : String
getUsername() : String
setPassword(password : String)
setUsername(username : String)
AdminUser
AdminUser(username : String)
Editor
level : PermissionLevel
Editor(username : String, level : PermissionLevel)
getLevel() : PermissionLevel
setLevel(level : PermissionLevel)
RegularUser
numberOfPosts : int
RegularUser(name : String)
getNumberOfPosts() : int
setNumberOfPosts(numberOfPosts : int)
-level
PermissionLevel
HIGH
LOW
MEDIUM
valueOf(name : String) : PermissionLevel
values() : PermissionLevel[]

Single Table

Opção A

- Espalmar (*flatten*) a hierarquia de classes (atributos) numa só tabela. Todas as subclasses estão presentes.
- Ter uma coluna que indica a classe do registo.

	123 level	123 number_of_posts	123 id	ABC dtype	ABC name	ABC password	ABC username
1	[NULL]	1	1	RegularUser	Serafim	benfica	Serafim
2	[NULL]	[NULL]	2	Person	Zacarias	[NULL]	[NULL]
3	[NULL]	[NULL]	3	AdminUser	Mestre André	benfica	Mestre André
4	0	[NULL]	4	Editor	Josefina	benfica	Josefina

Single Table

Opção A

- Espalmar (*flatten*) a hierarquia de classes (atributos) numa só tabela. Todas as subclasses estão presentes.
- Ter uma coluna que indica a classe do registo.

	123 level	123 number_of_posts	123 id	ABC dtype	ABC name	ABC password	ABC username
1	[NULL]	1	1	RegularUser	Serafim	benfica	Serafim
2	[NULL]	[NULL]	2	Person	Zacarias	[NULL]	[NULL]
3	[NULL]	[NULL]	3	AdminUser	Mestre André	benfica	Mestre André
4	0	[NULL]	4	Editor	Josefina	benfica	Josefina

Muitos NULLs!

Single Table

Opção A

- Espalmar (*flatten*) a hierarquia de classes (atributos) numa só tabela. Todas as subclasses estão presentes.
- Ter uma coluna que indica a classe do registo.

+ Acesso a atributos mais rápido

+ Simples de ler o schema

+ Flexível para mudança de atributos

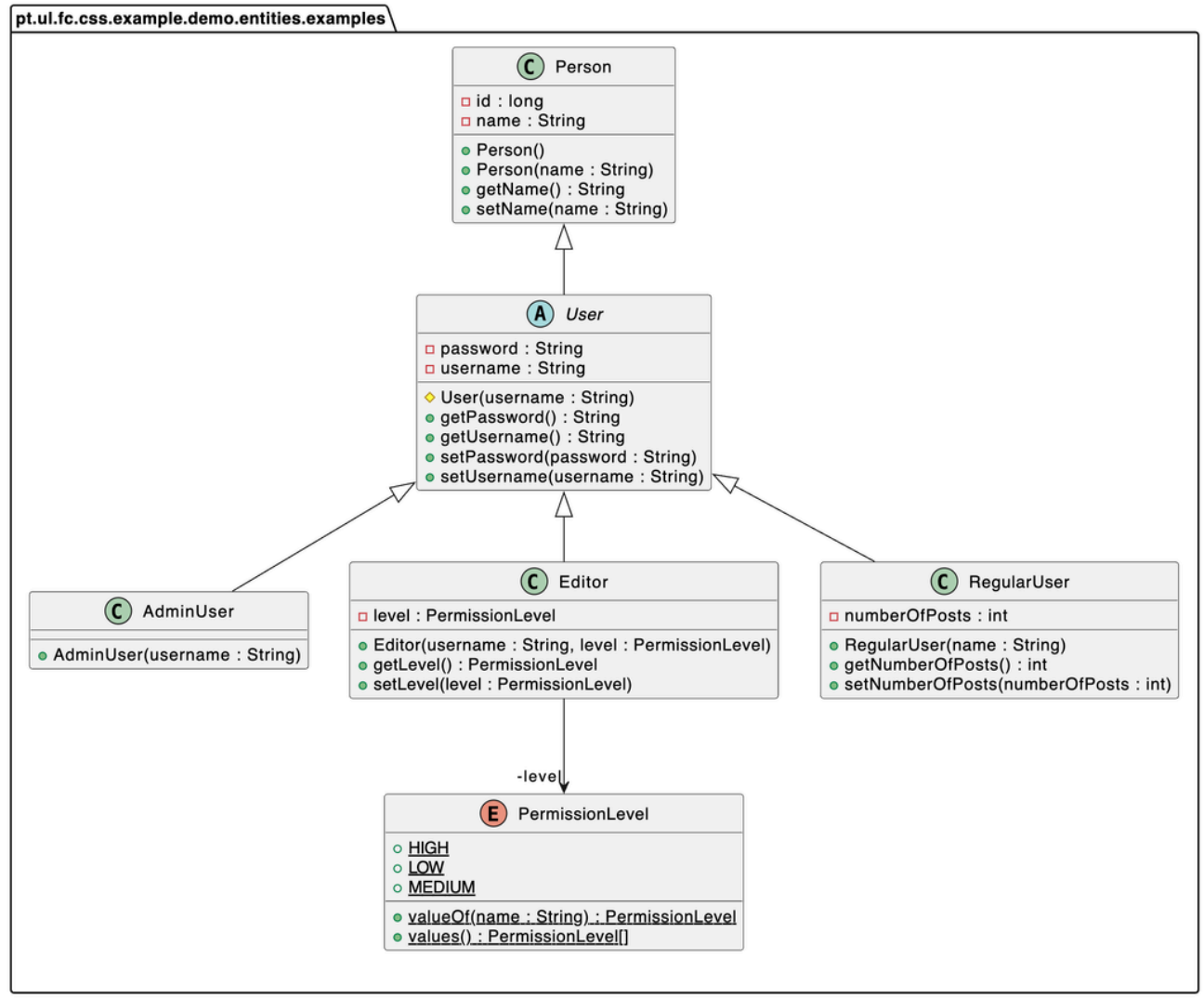
- Tabela muito esparsa (desperdício de espaço)

- Tabela muito larga para hierarquias grandes

Joined/Class Table

Opção B

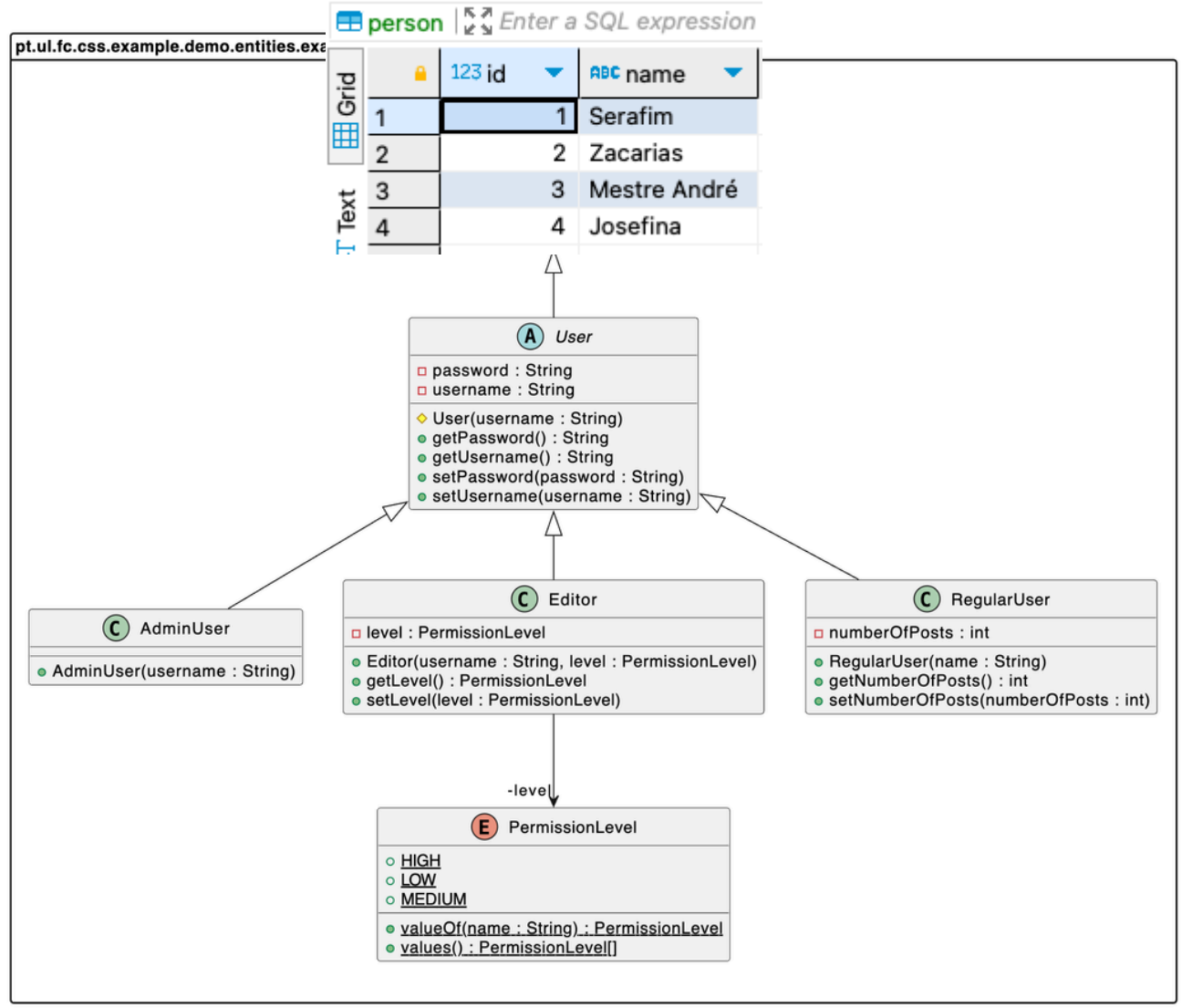
- Cada classe está numa tabela.
- Uma coluna por atributo não herdado.
- Uma coluna para o registo da superclasse.



Joined/Class Table

Opção B

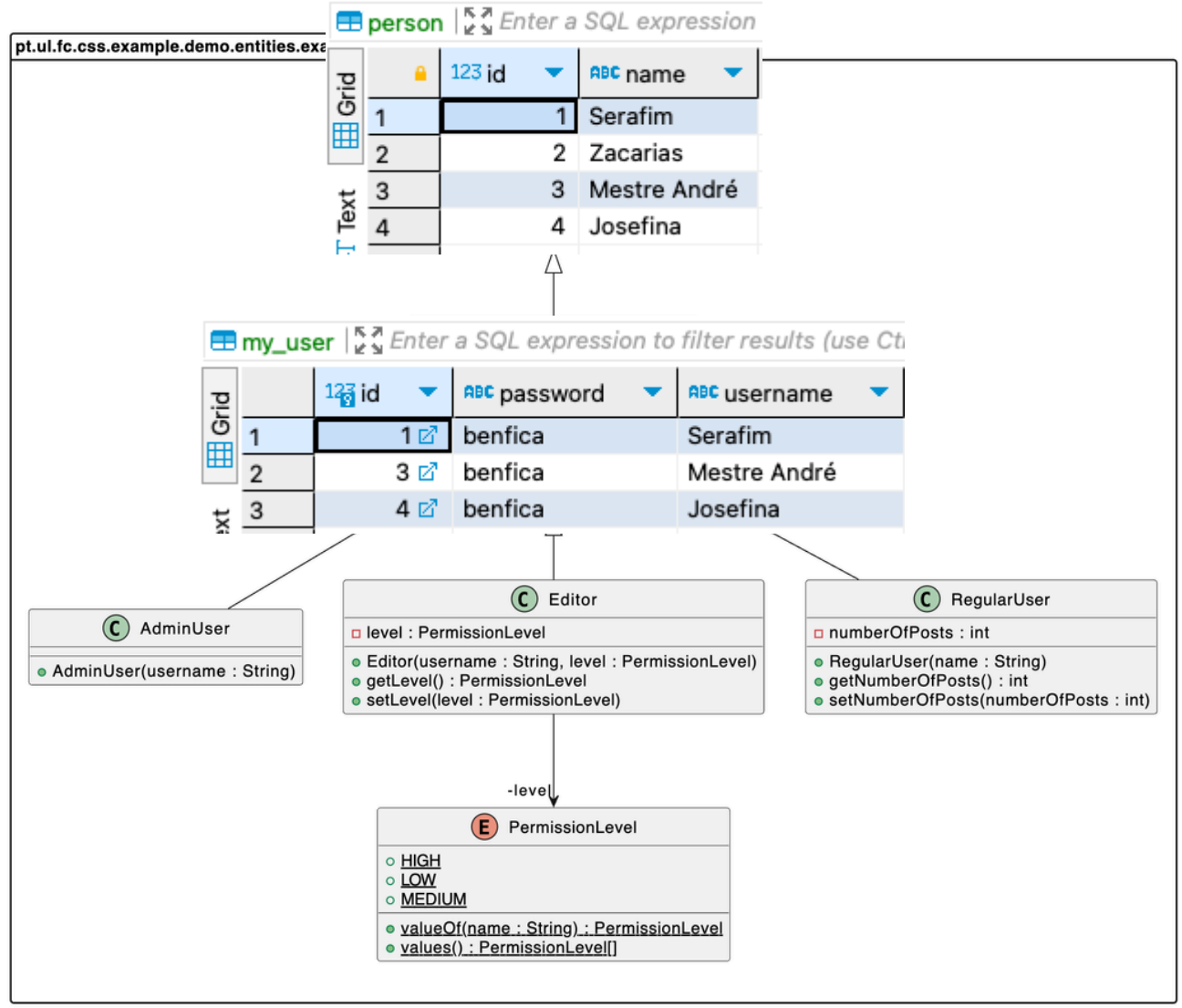
- Cada classe está numa tabela.
- Uma coluna por atributo não herdado.
- Uma coluna para o registo da superclasse.



Joined/Class Table

Opção B

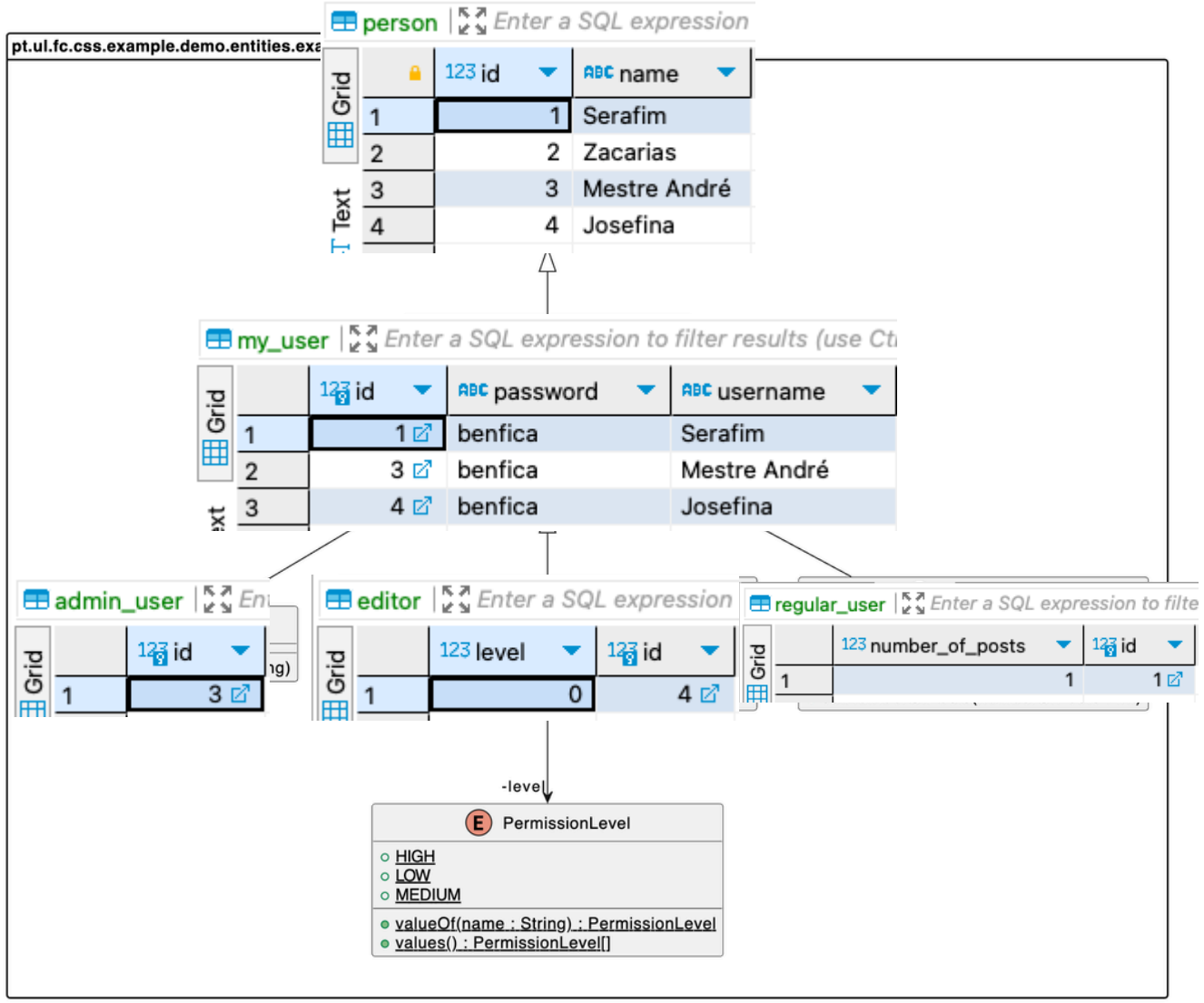
- Cada classe está numa tabela.
- Uma coluna por atributo não herdado.
- Uma coluna para o registo da superclasse.



Joined/Class Table

Opção B

- Cada classe está numa tabela.
- Uma coluna por atributo não herdado.
- Uma coluna para o registo da superclasse.



Joined/Class Table

Opção B

- Cada classe está numa tabela.
- Uma coluna por atributo não herdado.
- Uma coluna para o registo da superclasse.

person | Enter a SQL expression

	id	name
1	1	Serafim
2	2	Zacarias
3	3	Mestre André
4	4	Josefina

my_user | Enter a SQL expression to filter results (use Ct

	id	password	username
1	1	benfica	Serafim
2	3	benfica	Mestre André
3	4	benfica	Josefina

admin_user | En

	id
1	3

editor | Enter a SQL expression

	level	id
1	0	4

regular_user | Enter a SQL expression to filte

	number_of_posts	id
1	1	1

Joined/Class Table

Opção B

- Cada classe está numa tabela.
- Uma coluna por atributo não herdado.
- Uma coluna para o registo da superclasse.

+ Fácil de identificar a classe de cada atributo

+ Eficiente uso de espaço

+ Base de Dados normalizada

- Alterações implicam grandes alterações nos queries

- Acesso (muito!) lento a toda a informação das subclasses, à custa de JOINs entre tabelas.

Table per (Concrete) Class

Opção C

- Espalmar a hierarquia vertical numa tabela.
- Cada classe concreta vai ter uma tabela.
- Uma coluna por cada atributo, incluindo herdados.

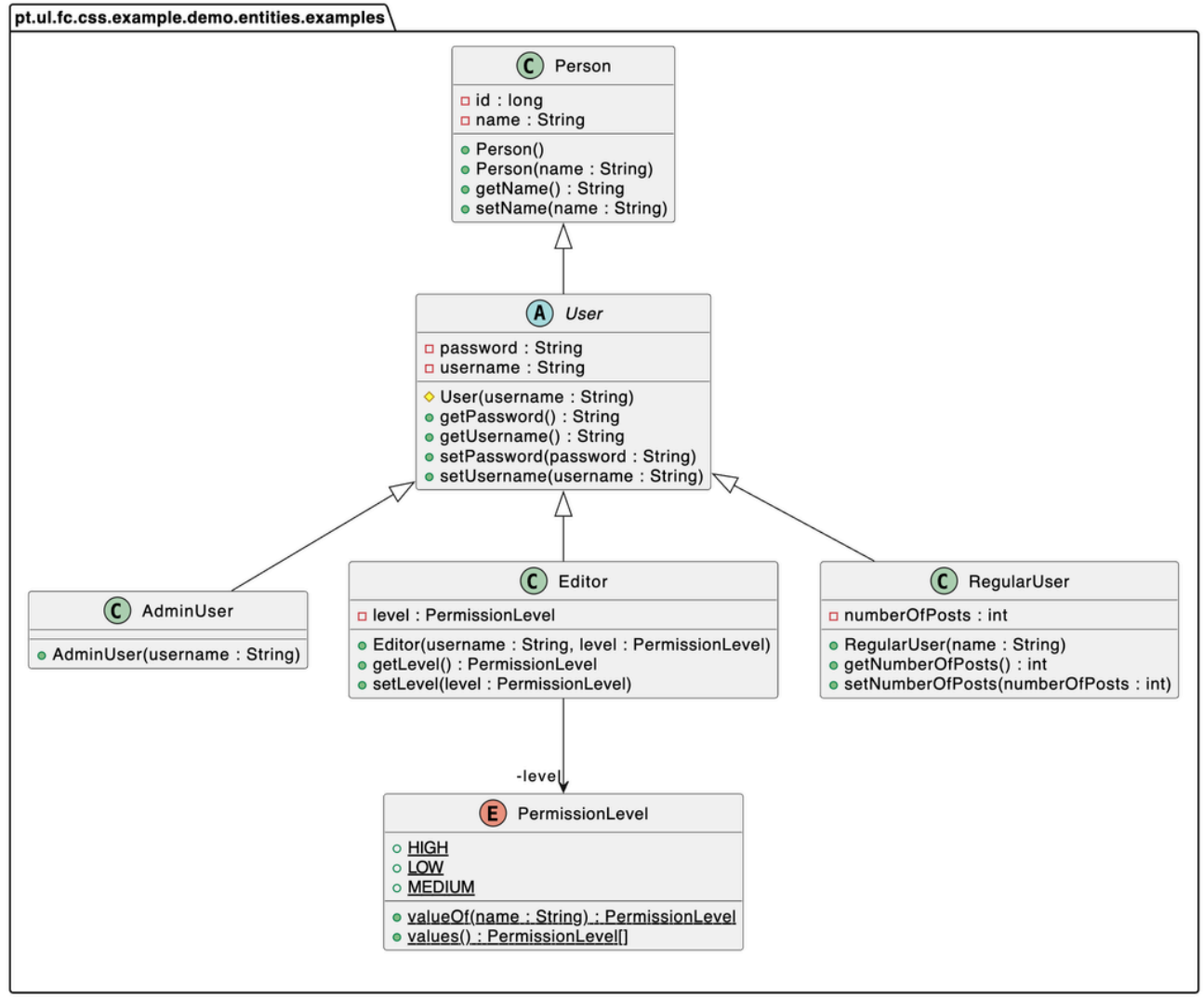


Table per (Concrete) Class

Opç

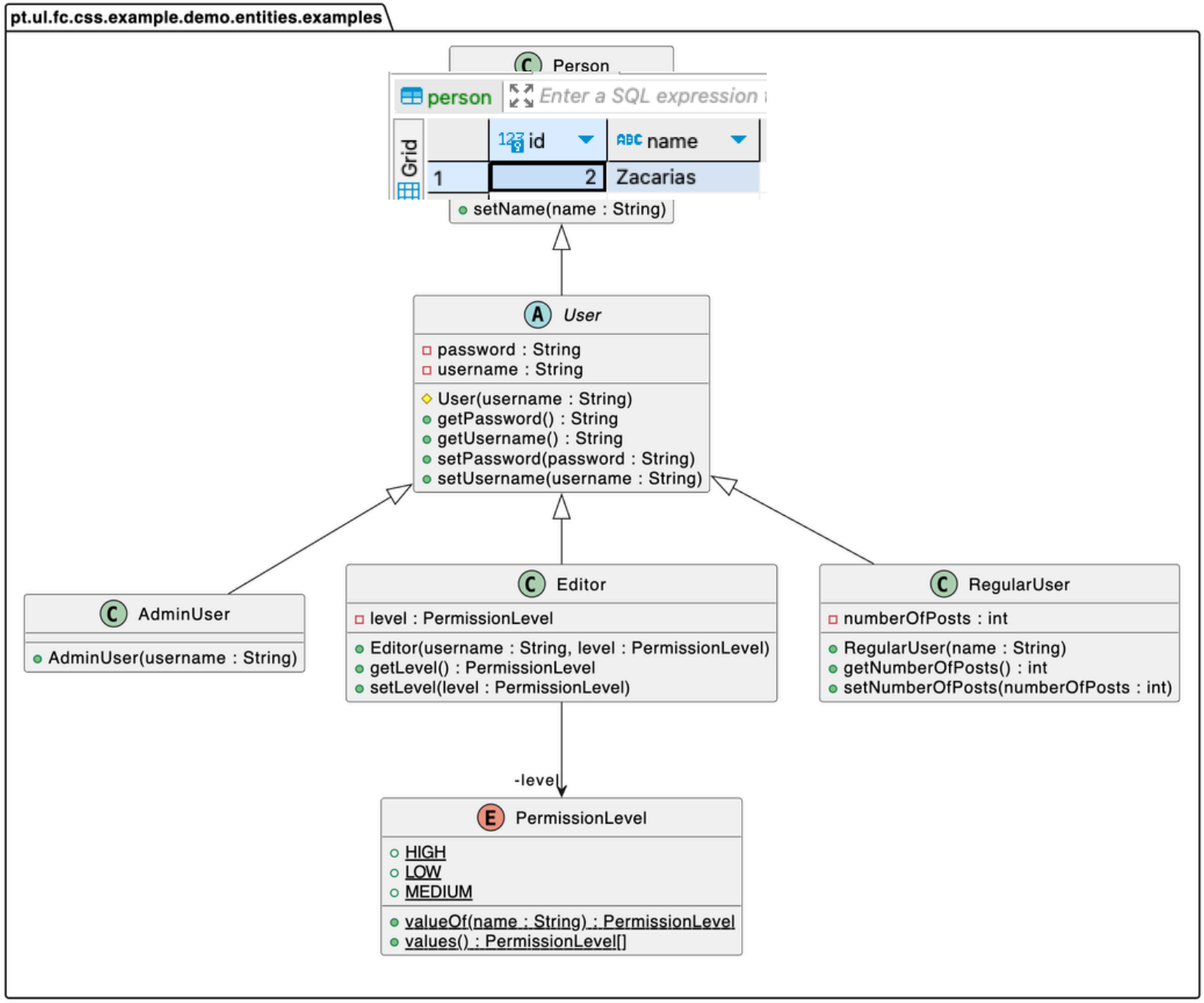


Table per (Concrete) Class

Opção C

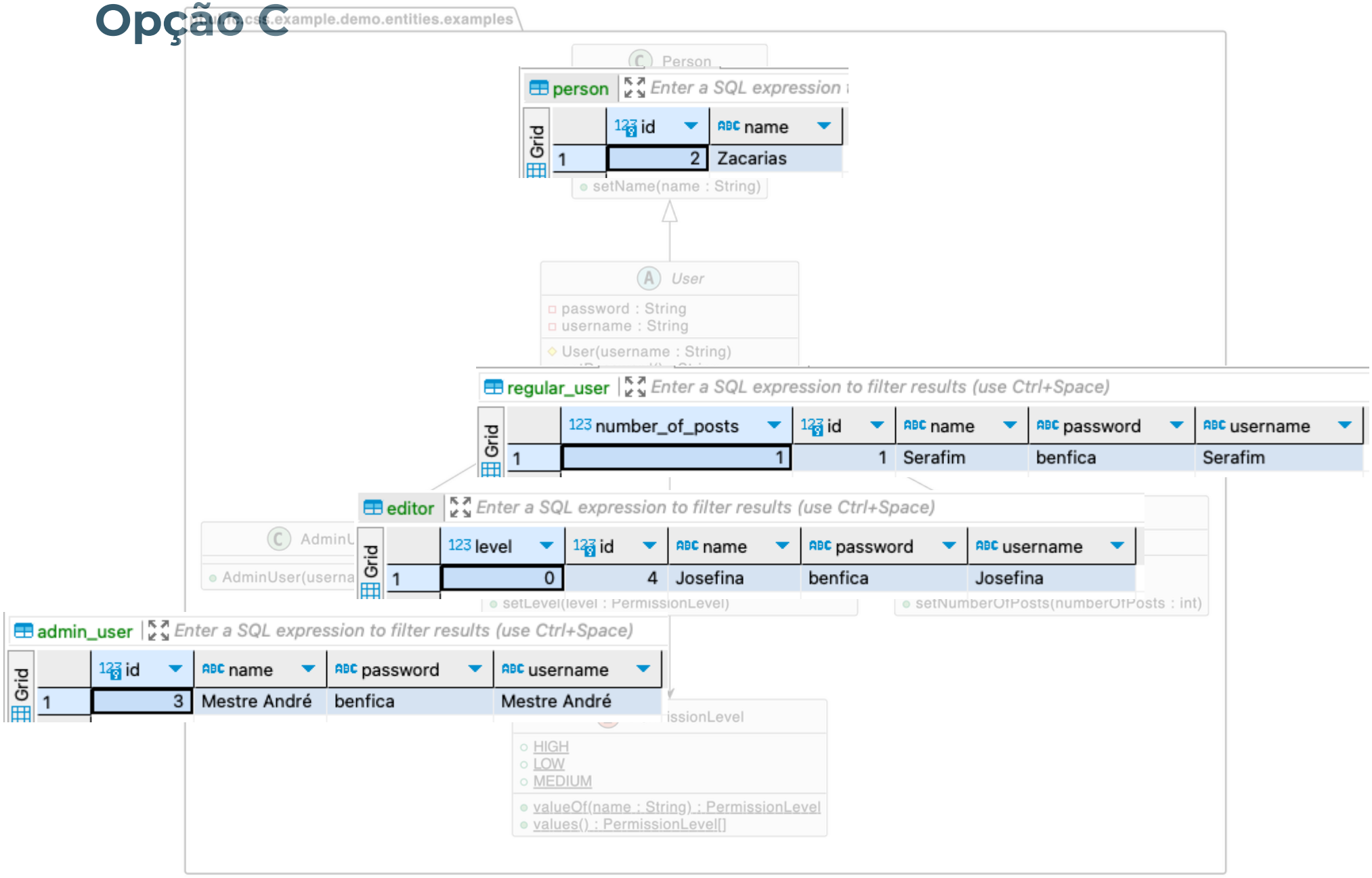


Table per (Concrete) Class

Opção C

- Espalmar a hierarquia vertical numa tabela.
- Cada classe concreta vai ter uma tabela.
- Uma coluna por cada atributo, incluindo herdados.

person | Enter a SQL expression

	id	name
1	2	Zacarias

regular_user | Enter a SQL expression to filter results (use Ctrl+Space)

	number_of_posts	id	name	password	username
1	1	1	Serafim	benfica	Serafim

editor | Enter a SQL expression to filter results (use Ctrl+Space)

	level	id	name	password	username
1	0	4	Josefina	benfica	Josefina

admin_user | Enter a SQL expression to filter results (use Ctrl+Space)

	id	name	password	username
1	3	Mestre André	benfica	Mestre André

Table per (Concrete) Class

Opção C

- Espalmar a hierarquia vertical numa tabela.
- Cada classe concreta vai ter uma tabela.
- Uma coluna por cada atributo, incluindo herdados.

+ Eficiente uso de espaço

+ Acesso rápido (sem joins!)

- Queries que relacionem com superclasses (abstractas ou não) são custosas. (e.g., listar todas as Persons ou todos os Users).

Table per (Concrete) Class

Opção C

```
select id, name, null as password, null as username, null as level,
null as number_of_posts, 0 as clazz_ from person union all

select id, name, password, username, null as level, null as
number_of_posts, 2 as clazz_ from admin_user union all

select id, name, password, username, level, null as number_of_posts,
3 as clazz_ from editor union all

select id, name, password, username, null as level, number_of_posts,
4 as clazz_ from regular_user
```

Mais recursos

https://www.baeldung.com/?s=JPA+relation

https://www.baeldung.com/hibernate-inheritance

https://openjpa.apache.org/builds/1.0.2/apache-openjpa-1.0.2/docs/manual/jpa_overview_mapping_inher.html

Object-Relational Mapping

Parte 1 - Entidades, Atributos Simples e Herança